KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM

(AN AUTONOMOUS INSTITUTION)

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

Kakapalayam (Po), Salem – 637 504

RECORD NOTE BOOK

REG. NO.

Certified that this is the bonafide record of work done by Selvan/Selvi …….…… of the **IV** Semester **B.Tech. Information Technology** Branch during the year **2025 - 2026** in the **BE23IT403 – Data Communication and Computer Networks** Laboratory.

Faculty – Incharge **Head of the Department**

Submitted for the model practical examination on ……..

Internal Examiner

CONTENTS

Ex. No.	Date	Name of The Experiment	Page No.	Mark Awarded	Signature
1		To Learn to use commands like tcpdump, netstat, ifconfig, nslookup and traceroute. Capture ping and traceroute PDUs using a network protocol analyzer and examine			
2		Write a client server program using socket			
3		Write a HTTP web client program to download a web page using TCP sockets.			
4		Use Wireshark to Capture Packets and examine the packets			
5		Simulation of Domain Name System (DNS Using UDP Sockets			
6(a)		Perform Address Resolution Protocol (ARP) using Network Simulator			
6(b)		Perform Reverse Address Resolution Protocol (RARP) using Network Simulator.			
7		Demonstrate Sliding Window Protocol.			
8		Demonstrate Subnetting within their ranges.			
9		Demonstrate working of DHCP with multiple client connection.			
10		Simulation of E-mail system focusing on sending, receiving, and deleting messages.			

Ex. No: 1	Networking Commands
Date:	

AIM

To learn about networking commands like tcpdump, netstat, ifconfig, nslookup and traceroute.

Networking Commands

1. **Netstat -** netstat (network statistics) is a command-line network utility that displays network connections for Transmission Control Protocol (both incoming and outgoing), routing tables, and a number of network interfaces (network interface controller or software-defined network interface) and network protocol statistics.

Command Prompt - netstat -a

```
Microsoft Windows [Version 10.0.15063]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\Aseem>netstat -a

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            AseemVostro:0          LISTENING
  TCP    0.0.0.0:443            AseemVostro:0          LISTENING
  TCP    0.0.0.0:445            AseemVostro:0          LISTENING
  TCP    0.0.0.0:902            AseemVostro:0          LISTENING
  TCP    0.0.0.0:912            AseemVostro:0          LISTENING
  TCP    0.0.0.0:2869           AseemVostro:0          LISTENING
  TCP    0.0.0.0:5357           AseemVostro:0          LISTENING
```

- **Netstat -a** – provides a list of all available TCP and UDP connections
- **Netstat -e** – displays details of packets that have been sent
- **Netstat -n** – lists currently connected hosts
- **Netstat -p** – allow to specify what type of protocol you want to check
- **Netstat -r** – provides a list of routing tables
- **Netstat -s** – gives statistics on IPv4, IPv6, ICMP, TCP, etc.

2. **Ipconfig -** ipconfig (standing for "Internet Protocol configuration") is a console application program of some computer operating systems that displays all current TCP/IP network configuration values and refreshes Dynamic Host Configuration Protocol (DHCP) and Domain Name System (DNS) settings.

Command Prompt

```
C:\Users\Aseem>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . : fios-router.home
   Link-local IPv6 Address . . . . . : fe80::257e:9cae:15eb:1922%10
   IPv4 Address. . . . . . . . . . . : 192.168.1.233
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
```

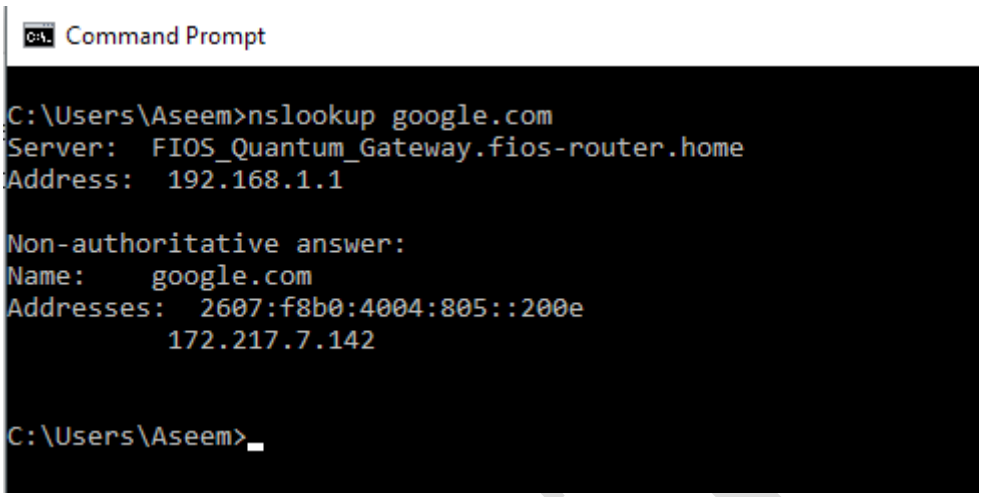
3. **Traceroute -** Traceroute is a command used in network troubleshooting for mapping the path packets travel through the network. The tool aids in the discovery of possible routes of information from source to destination. Additionally, the command also helps calculate the transfer times between points.

   When applied to network troubleshooting, traceroute helps locate where traffic slows down between the source and destination.

Command Prompt - tracert helpdeskgeek.com

```
C:\Users\Aseem>tracert helpdeskgeek.com

Tracing route to helpdeskgeek.com [52.5.121.7]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  FIOS_Quantum_Gateway.fios-router.home [192.
  2     1 ms     2 ms     2 ms  lo0-100.BLTMMD-VFTTP-315.verizon-gni.net [1
  3     7 ms     7 ms     7 ms  B3315.BLTMMD-LCR-22.verizon-gni.net [130.81
  4     *        *        *     Request timed out.
  5     *        *        *     Request timed out.
  6     6 ms     6 ms     6 ms  0.et-11-0-2.GW13.IAD8.ALTER.NET [140.222.0.
  7     7 ms     5 ms     6 ms  neustar-gw.customer.alter.net [152.179.50.5
  8     *        *        *     Request timed out.
  9     *        *        *     Request timed out.
 10     5 ms    12 ms    19 ms  54.239.110.179
```

4. **NsLookup -** is a utility to query DNS tables, using this utility you can find what your DNS server is or any DNS server you specify, for example, nslookup google.com



5. **Tcpdump**

tcpdump -i eth0 icmp

which will list ping traffic on interface eth0.

:~$ **sudo tcpdump -n icmp**

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode listening on enp7s0, link-type EN10MB (Ethernet), capture size 262144 bytes

11:34:21.590380 IP 10.10.1.217 > 10.10.1.30: ICMP echo request, id 27948, seq 1, length 64

11:34:21.590434 IP 10.10.1.30 > 10.10.1.217: ICMP echo reply, id 27948, seq 1, length 64

11:34:27.680307 IP 10.10.1.159 > 10.10.1.1: ICMP 10.10.1.189 udp port 59619 unreachable, length 115

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, the various networking commands have been learnt and executed successfully.

Ex. No: 2	Write a client server program using socket
Date:	

Aim:

To create a simple client-server application using sockets in Java. The client will send a message to the server, and the server will respond by echoing the message back to the client.

Program:

Server Code (Server.java)

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(12345)) {
            System.out.println("Server is listening on port 12345...");

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");
                BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

                    String message = input.readLine();
                System.out.println("Received from client: " + message);


                output.println("Echo: " + message);

                        socket.close();
                System.out.println("Client disconnected");
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }

}}
```

Client Code (Client.java)

```
import java.io.*;
import java.net.*;
```

```
public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 12345)) {
            System.out.println("Connected to server");

                 BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

                String message = "Hello, Server!";
            output.println(message);
            System.out.println("Sent to server: " + message);

              String response = input.readLine();
            System.out.println("Received from server: " + response);
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

Output:

1. Server Output:

```
Server is listening on port 12345...
New client connected
Received from client: Hello, Server!
Client disconnected
```

2. Client Output:

```
Connected to server
Sent to server: Hello, Server!
Received from server: Echo: Hello, Server!
```

KIOT

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, a HTTP web client program to download a webpage using TCP sockets has been written and executed successfully.

Ex. No: 3	Write a HTTP web client program to download a web page using TCP sockets.
Date:	

Aim:

To write a HTTP web client program to download a webpage using TCP sockets.

Algorithm:

Server Side

Step 1. Start.
Step 2. Import the necessary packages.
Step 3. Accept the request from the client.
Step 4. Establish socket connection between client and server.
Step 5. Read the image and send it to the client.
Step 6. Close the socket connection.
Step 7. Stop.

Client Side

Step 1. Start.
Step 2. Import the necessary packages.
Step 3. Establish the socket connection between client and server. Step 4. Get the image from the server.
Step 5. Display the size of the image. Step 6. Close the socket connection. Step 7. Stop.

Program:

Server1.java

```
import java.net.*;
import java.io.*;
import java.awt.image.*;
import javax.imageio.*;
```

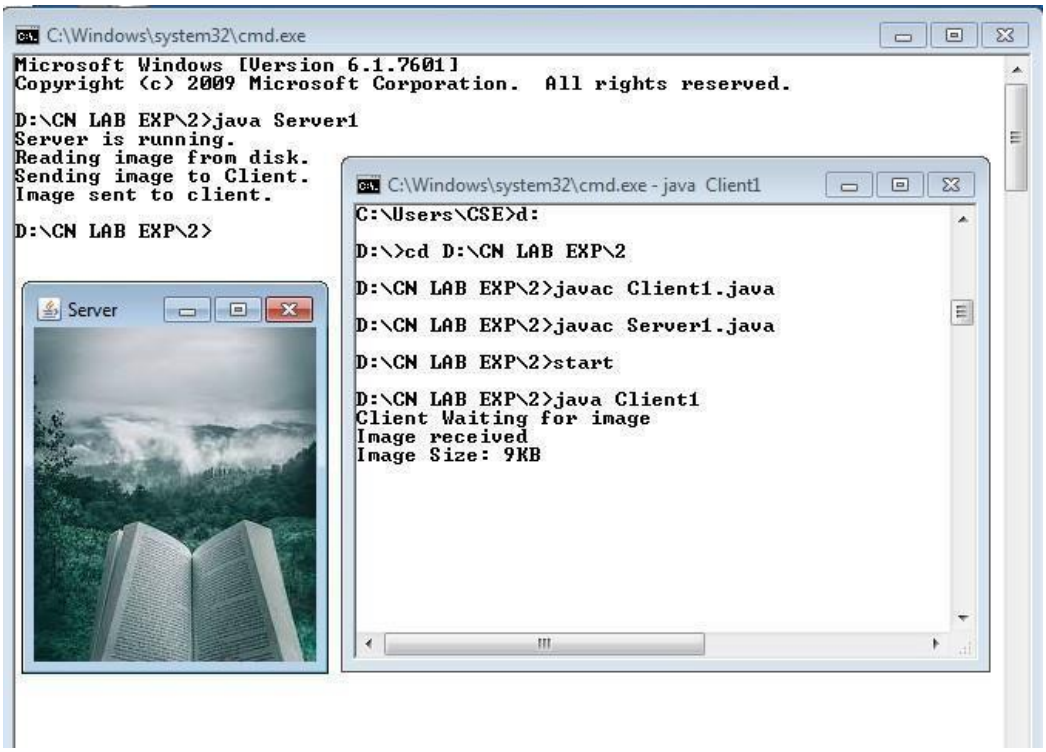
```
import javax.swing.*;

import  java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import
javax.imageio.ImageIO;


public class Server1
{
public static void main(String args[]) throws Exception
{
ServerSocket server=null;
Socket socket;
BufferedImage img =
null;
server=new ServerSocket(4000);
System.out.println("Server is running. ");
socket=server.accept();
try
{
System.out.println("Reading image from disk.
"); img = ImageIO.read(new
File("image1.jpg"));
ByteArrayOutputStream baos = new
ByteArrayOutputStream(); ImageIO.write(img, "jpg", baos);
baos.flush();
byte[] bytes = baos.toByteArray();
baos.close();
System.out.println("Sending image to Client. ");
OutputStream out = socket.getOutputStream();
DataOutputStream dos = new
DataOutputStream(out); dos.writeInt(bytes.length);
dos.write(bytes, 0, bytes.length);
System.out.println("Image sent to client.
"); dos.close();
out.close();
}
catch(Exception e)
```

```
{
System.out.println("Exception: " + e.getMessage());
socket.close();
}
socket.close();
}
}
```

Client1.java

```
import java.net.*;
import java.io.*;
import java.awt.image.*;
import  javax.imageio.*;
import javax.swing.*;
public class Client1
{
public static void main(String args[]) throws Exception
{
Socket soc;
soc=new Socket("localhost",4000);
System.out.println("Client Waiting for image");
InputStream in = soc.getInputStream();
DataInputStream dis = new DataInputStream(in);
int len = dis.readInt();
System.out.println("Image received");
System.out.println("Image Size: " + len/1024 + "KB");
byte[] data = new byte[len];
dis.readFully(data);
dis.close();
in.close();
InputStream ian = new ByteArrayInputStream(data);
BufferedImage bImage = ImageIO.read(ian);
JFrame f = new JFrame("Server");
ImageIcon icon = new
ImageIcon(bImage); JLabel l = new
JLabel();
l.setIcon(icon);
f.add(l);
f.pack();
f.setVisible(true);
}}
```

Output



Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, a HTTP web client program to download a webpage using TCP sockets has been written and executed successfully.

Ex. NO: 4	Use a tool like Wireshark to Capture Packets and examine the packets
Date:	

Aim:

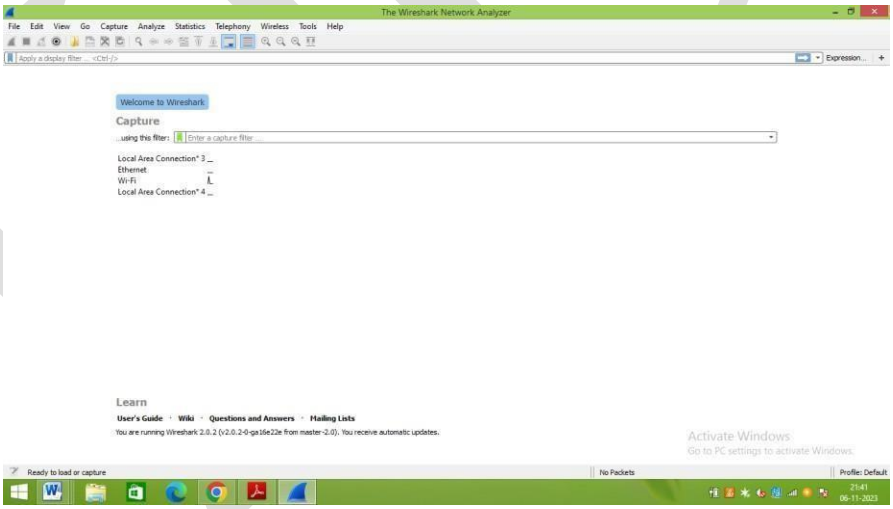
To capture and examine the packets using wireshark tool.

Theory

Wireshark

Wireshark is a free open- source network protocol analyzer. It is used for network troubleshooting and communication protocol analysis. Wireshark captures network packets in real time and display them in human-readable format. It provides many advanced features including live capture and offline analysis, three-pane packet browser, coloring rules for analysis.

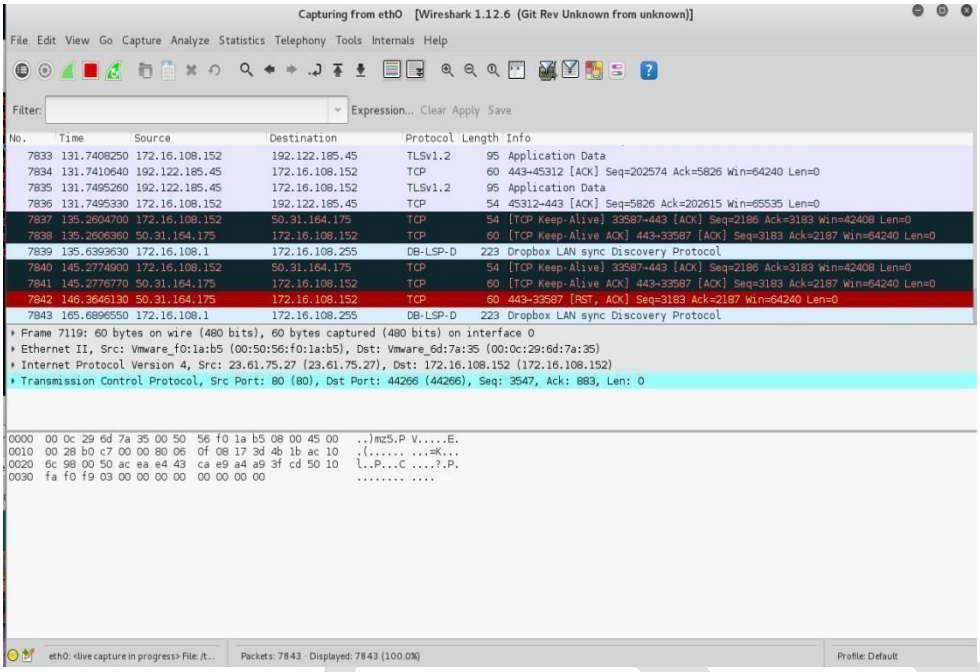
- Start up the WiresStart up the Wireshark program (select an interface and press start to capture packets).



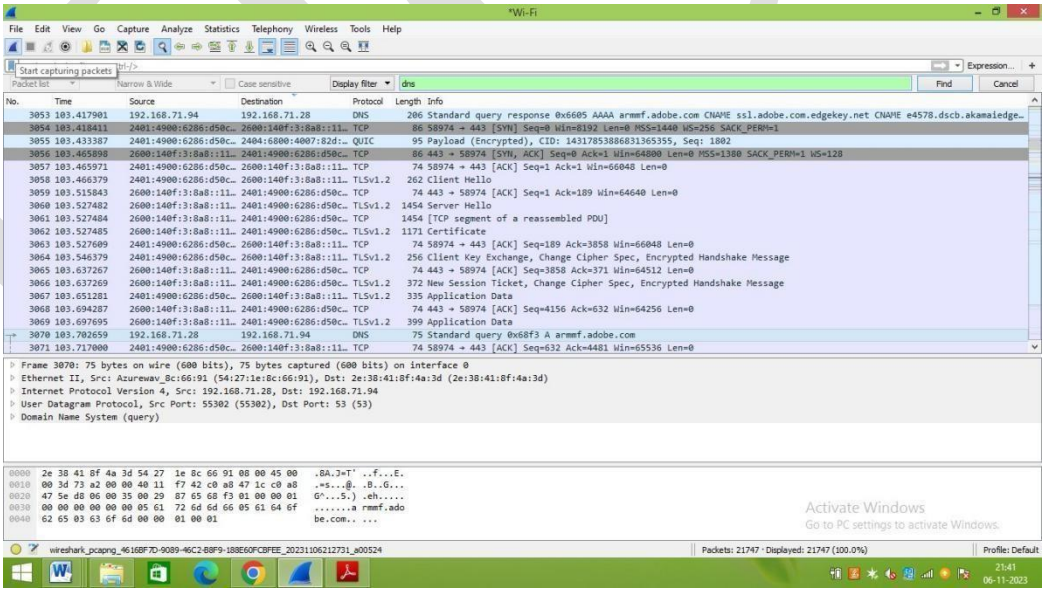
- Start up the Wireshark program (select an interface and press start to capture packets).

- Start up your favorite browser
- After your browser has displayed the page, stop Wireshark packet capture by selecting stop in

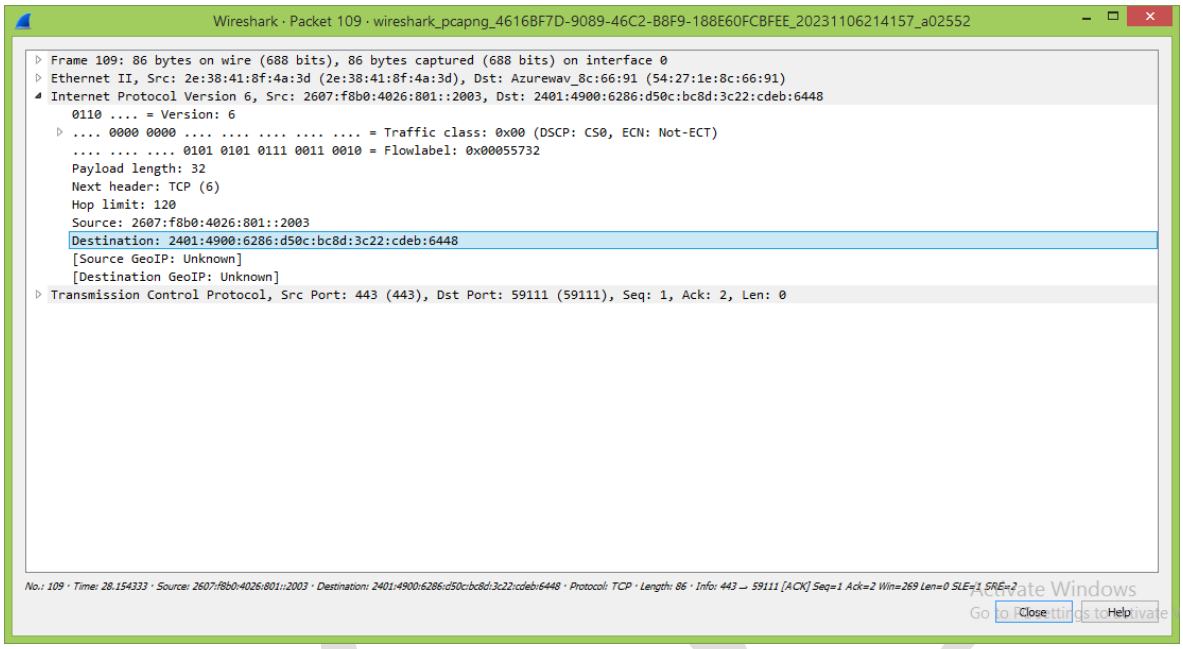
the Wireshark capture window. This will cause the Wireshark capture window to disappear and the main Wireshark window to display all packets captured



Here tcp packets and frame formats are displayed. In this window sequence number, acknowledgement number , header length, window size, sequence Ack no.

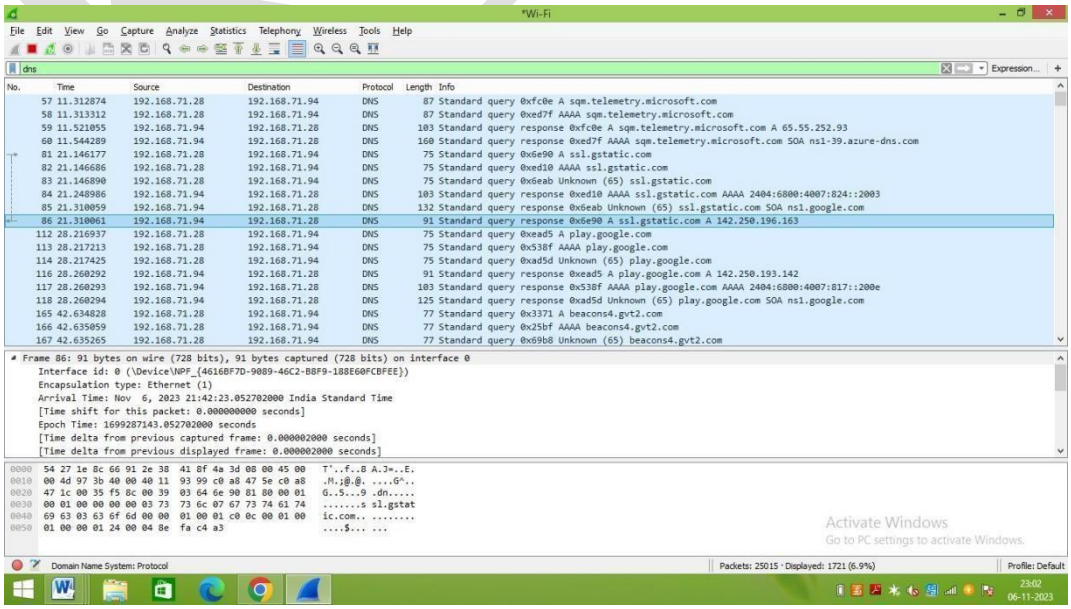


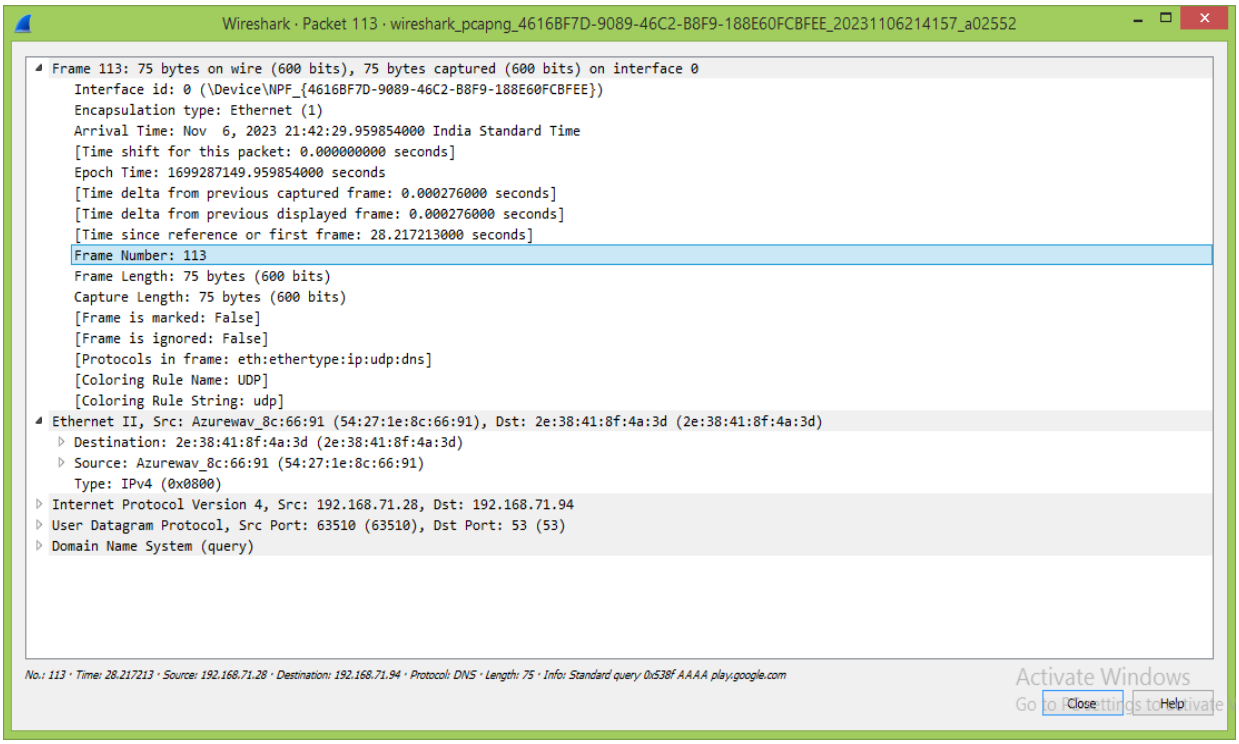
Here is the format of ipV6 protocol in tcp.



Domain Name System

Domain Name System is the system used to resolve store information about domain names including IP addresses, mail servers, and other information.





Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result

Thus captured and examined the packets using wireshark tool.

Ex. No:5	Simulation of DNS using UDP sockets.
Date:	

Aim:

To write a Java program for simulation of DNS using UDP sockets.

Algorithm:

Server Side

Step 1. Start.
Step 2. Create UDP datagram socket.
Step 3. Create a table that maps host name and IP address.
Step 4. Receive the host name/IP address from the client.
Step 5. Retrieve the client's IP address/host name from the received datagram.
Step 6. Get the IP address/host name mapped from the table.
Step 7. Display the host name and corresponding IP address.
Step 8. Send the IP address/host name to the client.
Step 9. Stop.

Client

Step 1. Start.
Step 2. Create UDP datagram socket.
Step 3. Get the host name from the client. Step 4. Send the host name to the server. Step 5. Wait for the reply from the server.
Step 6. Receive the reply datagram and read the IP address for the requested host name.
Step 7. Display the IP address.
Step 8. Stop.

Program:

<u>Serverdns12.java</u>

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverdns12
{
public static void main(String args[])
{
try
{
```

```
  System.out.println("Server Ready");
  System.out.println("Press Ctrl + C to Quit");
  DatagramSocket server=new DatagramSocket(1309);
  while(true)

  {byte[] sendbyte=new byte[1024];
byte[] receivebyte=new byte[1024];
  DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
  server.receive(receiver);
  String str=new String(receiver.getData());
  String s=str.trim();
  InetAddress addr=receiver.getAddress();
  int port=receiver.getPort();
  String ip[]={"165.165.80.80","165.165.79.1"};
  String name[]={"www.aptitudeguru.com","www.downloadcyclone.blogspot.com"};
  for(int i=0;i<ip.length;i++)
  {
  if(s.equals(ip[i]))
  {
  sendbyte=name[i].getBytes();
  DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,port);
  server.send(sender);
  break;
  }
  else if(s.equals(name[i]))
  {
  sendbyte=ip[i].getBytes();
  DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,port);
  server.send(sender);
  break;
  }
  }
  break;
  }
  }
  catch(Exception e)
  {
  System.out.println(e);
  }
  }
  }
```

Client

dns12.java

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientdns12
{
public static void main(String args[])
{try

{
DatagramSocket client=new DatagramSocket();
InetAddress
addr=InetAddress.getByName("127.0.0.1"); byte[]
sendbyte=new byte[1024];
byte[] receivebyte=new byte[1024];
BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter the DOMAIN NAME or IP adress:");
String str=in.readLine();
sendbyte=str.getBytes();
DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,1309);
client.send(sender);
DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
client.receive(receiver);
String s=new String(receiver.getData());
System.out.println("IP address or DOMAIN NAME:
"+s.trim()); client.close();
}
catch(Exception e)
{
System.out.println(e);
}
}
}
```

Output:

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, a Java program simulation of DNS using UDP sockets has been written and executed successfully.

Ex. No: 6(a)	Perform Address Resolution Protocol (ARP) using Network Simulator.
Date:	

Aim:

To write a Java program for simulating ARP protocol using TCP.

Algorithm:

Server Side

Step 1. Start.
Step 2. Create a server socket and bind it to port.
Step 3. Listen for new connection and when a connection arrives, accept it. Step 4. Read the logical address sent by the client.
Step 5. Send the corresponding physical address to the client.
Step 6. Close the server socket.
Step 7.
Stop.

Client Side

Step 1.
Start.
Step 2. Create a client socket and connect it to the server. Step 3. Send the logical address to the server.
Step 4. Display the corresponding address sent by the server.
Step 5. Close the input and output streams.
Step 6. Close the client socket.
Step 7. Stop.

Program:

<u>Serverrarp.java</u>

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverarp
{
public static void main(String args[])
{
try
{
System.out.println("Server Ready");
```

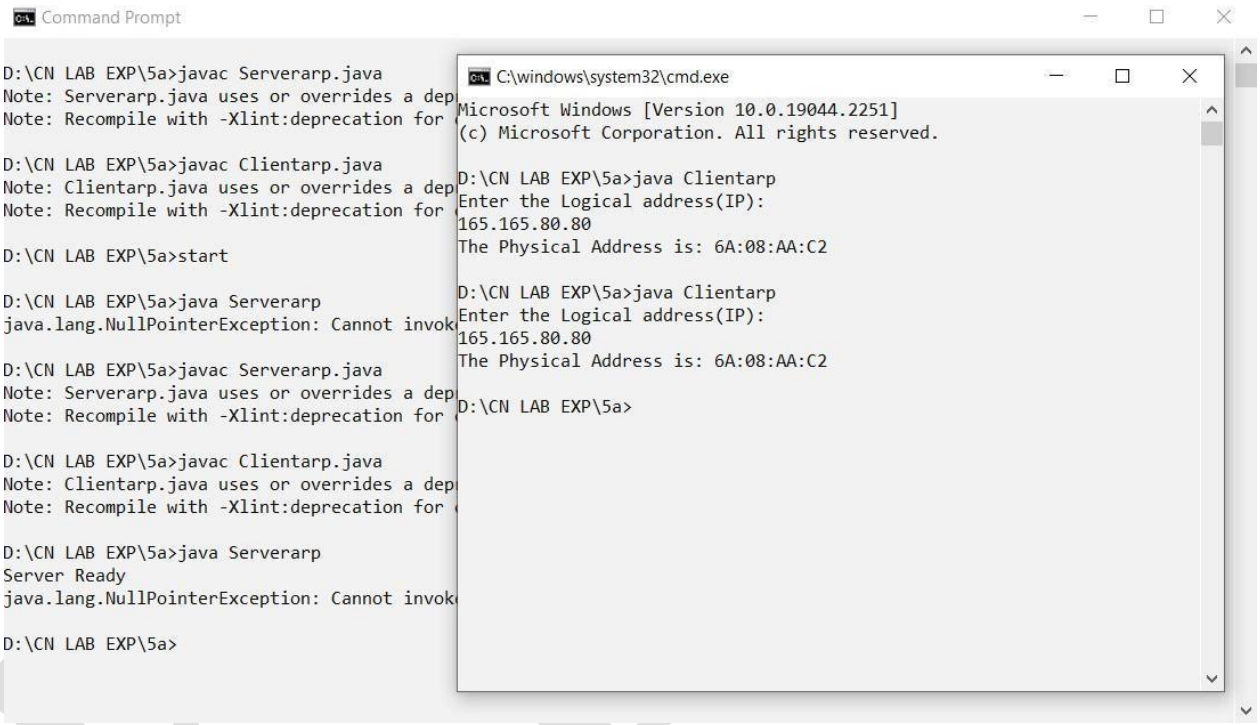
```
ServerSocket obj=new
ServerSocket(5000); Socket
obj1=obj.accept();
while(true)
{
DataInputStream din=new DataInputStream(obj1.getInputStream());
DataOutputStream dout=new DataOutputStream(obj1.getOutputStream());
String str=din.readLine();
String ip[]={"165.165.80.80","165.165.79.1"};
String mac[]={"6A:08:AA:C2","8A:BC:E3:FA"};
for(int i=0;i<ip.length;i++)
{
if(str.equals(ip[i]))
{
dout.writeBytes(mac[i]+'\n');
break;
}
}
obj.close();
}
}
catch(Exception e)
{
System.out.println(e);
}
}
}
```

Clientarp.java

```
import java.io.*;
import
java.net.*;
import
java.util.*; class
Clientarp
{
public static void main(String args[])
{
try
{
```

```
BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
Socket clsct=new Socket("127.0.0.1",5000);
DataInputStream din=new DataInputStream(clsct.getInputStream());
DataOutputStream dout=new DataOutputStream(clsct.getOutputStream());
System.out.println("Enter the Logical address(IP):");
String str1=in.readLine();
dout.writeBytes(str1+'\n');
String str=din.readLine();
System.out.println("The Physical Address is: "+str);
clsct.close();
}
catch (Exception e)
{
System.out.println(e);
}
}
}
```

Output:



Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, a Java program to simulate ARP protocols using TCP has been written and executed successfully.

Ex. No: 6(b)	**Perform Reverse Address Resolution Protocol (RARP) using Network Simulator.**
Date:	

Aim:

To write a Java program for simulating RARP protocol using UDP.

Algorithm:

Server Side

Step 1. Start.

Step 2. Server maintains the table in which IP and corresponding MAC addresses are stored.

Step 3. Create the datagram socket

Step 4. Receive the datagram sent by the client and read the MAC address sent. Step 5. Retrieve the IP address for the received MAC address from the table.

Step 6. Display the corresponding IP address.

Step 7. Stop

Client Side

Step 1. Start the program

Step 2. Create datagram socket

Step 3. Get the MAC address to be converted into IP address from the user. Step 4. Send this MAC address to server using UDP datagram.

Step 5. Receive the datagram from the server and display the corresponding IP address. Step 6. Stop

Program:

Serverap12.java

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverrarp12
{
public static void main(String args[])
{

try
{
System.out.println("Server Ready");
DatagramSocket server=new
```

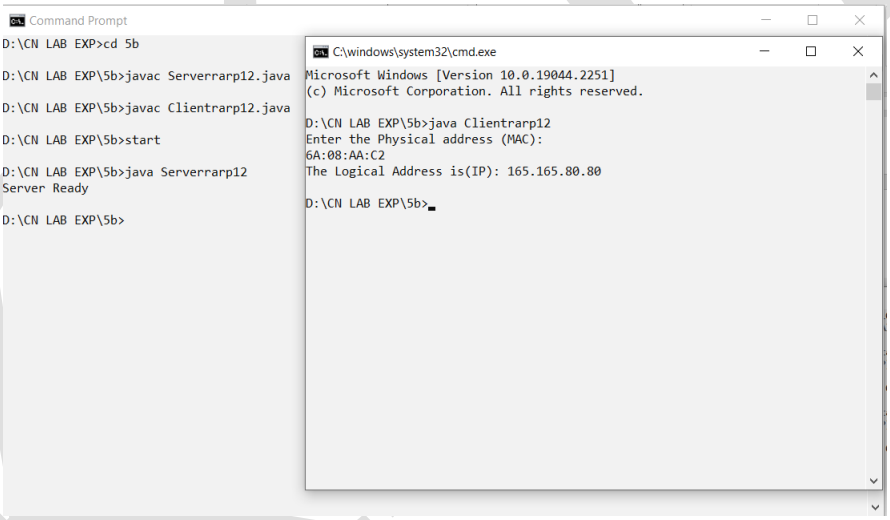
```
DatagramSocket(1309); while(true)
{
byte[] sendbyte=new byte[1024];
byte[] receivebyte=new byte[1024];
DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
server.receive(receiver);
String str=new String(receiver.getData());
String s=str.trim();
InetAddress addr=receiver.getAddress();
int port=receiver.getPort();
String ip[]={"165.165.80.80","165.165.79.1"};
String mac[]={"6A:08:AA:C2","8A:BC:E3:FA"};
for(int i=0;i<ip.length;i++)
{
if(s.equals(mac[i]))
{
sendbyte=ip[i].getBytes();
DatagramPacket sender = new
DatagramPacket(sendbyte,sendbyte.length,addr,port);
server.send(sender);
break;
}
}
break;
}
}
catch(Exception e)
{
System.out.println(e);
}
}
}
```

Clientrarp12.java

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientrarp12
{
public static void main(String args[])
{
try
```

```
{
DatagramSocket client=new DatagramSocket();
InetAddress
addr=InetAddress.getByName("127.0.0.1"); byte[]
sendbyte=new byte[1024];
byte[] receivebyte=new byte[1024];
BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter the Physical address (MAC):");
String str=in.readLine(); sendbyte=str.getBytes();
DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,1309);
client.send(sender);
DatagramPacket receiver=new DatagramPacket(receivebyte,receivebyte.length);
client.receive(receiver);
String s=new String(receiver.getData());
System.out.println("The Logical Address is(IP): "+s.trim());
client.close();
}
catch(Exception e)
{
System.out.println(e);
}
}
}
```

Output:



Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

Thus, a Java program to simulate RARP protocols using UDP has been written and executed

successfully.

Ex. No: 7	Demonstrate Sliding Window Protocol.
Date:	

Aim:

To demonstrate the Sliding Window Protocol, which ensures reliable and efficient data transmission between sender and receiver.

Algorithm:

1. Sender Side:

- Divide data into fixed-size frames.
- Assign sequence numbers to each frame.
- Maintain a sending window (range of frames that can be sent without acknowledgment).
- Send frames within the window.
- Wait for acknowledgments (ACKs) from the receiver.
- Slide the window forward upon receiving ACKs.

2. Receiver Side:

- Receive frames and check for errors.
- Send ACKs for correctly received frames.
- Maintain a receiving window (range of frames expected).
- Deliver frames in order to the upper layer.

Program:

```
import java.util.*;

class SlidingWindow {
    private int windowSize;
    private List<Integer> senderWindow;
    private  List<Integer>  receiverBuffer;
    private List<Integer> ackReceived;

    public SlidingWindow(int windowSize) {
        this.windowSize = windowSize;
        this.senderWindow = new ArrayList<>();
        this.receiverBuffer = new ArrayList<>();
        this.ackReceived = new ArrayList<>();
    }

    public List<Integer> sendFrames(List<Integer> frames) {
```

```
        System.out.println("Sender: Sending frames: " + frames.subList(0, windowSize));
        senderWindow = frames.subList(0, windowSize);
        return senderWindow;
    }

    public List<Integer> receiveFrames(List<Integer> frames) {
        System.out.println("Receiver: Receiving frames: " + frames);
        receiverBuffer.addAll(frames);

        for (int i = 0; i < frames.size(); i++) {
            ackReceived.add(i);
        }
        return ackReceived;
    }
    public void slideWindow(int ack) {
            System.out.println("Sender: Sliding window after ACK: " + ack);
            senderWindow = senderWindow.subList(ack + 1, senderWindow.size());
        }

        public static void main(String[] args) {
            List<Integer> frames = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
            int windowSize = 3;
            SlidingWindow protocol = new SlidingWindow(windowSize);
                    List<Integer> sentFrames = protocol.sendFrames(frames);
            List<Integer> acks = protocol.receiveFrames(sentFrames);
            protocol.slideWindow(acks.get(acks.size() - 1));
        }
    }
```

Output:

Sender: Sending frames: [1, 2, 3]
Receiver: Receiving frames: [1, 2, 3]
Sender: Sliding window after ACK: 2

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

The sender successfully sends frames, the receiver acknowledges them, and the sender slides the window forward.

Ex. No: 8	Demonstrate Subnetting within their ranges.
Date:	

.

Aim:

To create a simple client-server application using sockets in Java. The client will send a message to the server, and the server will respond by echoing the message back to the client.

Algorithm:

1. Choose an IP address and its default subnet mask.
2. Determine the number of subnets and hosts required.
3. Calculate the new subnet mask based on the requirements.
4. Divide the network into subnets using the new subnet mask.
5. Assign IP ranges to each subnet.

Program:

Server Code (Server.java):

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(12345)) {
            System.out.println("Server is listening on port 12345...");

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");

                        BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

                        String message = input.readLine();
                System.out.println("Received from client: " + message);

                    output.println("Echo: " + message);
```

```
            socket.close();
            System.out.println("Client disconnected");
        System.out.println("Client disconnected");
          }
      } catch (IOException ex) {
          ex.printStackTrace();
      }
  }
}
```

Client Code (Client.java):

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 12345)) {
            System.out.println("Connected to server");

            // Input and output streams
            BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

            // Send a message to the server
            String message = "Hello, Server!";
            output.println(message);
            System.out.println("Sent to server: " + message);

            // Receive response from the server
            String response = input.readLine();
            System.out.println("Received from server: " + response);
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

Output:

1. Server Output:

```
Server is listening on port 12345...
New client connected
Received from client: Hello, Server!
Client disconnected
```

2. Client Output:

```
Connected to server
Sent to server: Hello, Server!
Received from server: Echo: Hello, Server!
```

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

The network is divided into 4 subnets, each with 62 usable host addresses.

Ex. No: 9	Demonstrate working of DHCP with multiple client connection.
Date:	

Aim:

To demonstrate how DHCP dynamically assigns IP addresses to multiple clients in a network.

Algorithm:

1. DHCP Server Setup:
   - Configure a pool of available IP addresses.
   - Set lease duration for IP addresses.

2. Client Request:
   - Client sends a DHCP Discover message (broadcast).
   - DHCP server responds with a DHCP Offer message (unicast).

3. IP Assignment:
   - Client sends a DHCP Request message (broadcast).
   - DHCP server sends a DHCP Acknowledge message (unicast) with the assigned IP.

4. Lease Renewal:
   - Client renews the lease before it expires.

Program:

```
import java.util.*;
class DHCPServer {
    private Set<String> ipPool;
    private Map<String, String> assignedIps;

    public DHCPServer() {
        ipPool = new HashSet<>(Arrays.asList("192.168.1.10", "192.168.1.11", "192.168.1.12"));
        assignedIps = new HashMap<>();
    }

    public String assignIp(String clientId) {
        if (!ipPool.isEmpty()) {
```

```
        String ip = ipPool.iterator().next();
        ipPool.remove(ip);
        assignedIps.put(clientId, ip);
        return ip;
    }
    return null;
}

public void releaseIp(String clientId) {
    if (assignedIps.containsKey(clientId)) {
        String ip = assignedIps.remove(clientId);
        ipPool.add(ip);
    }
}
  public void releaseIp(String clientId) {
      if (assignedIps.containsKey(clientId)) {
          String ip = assignedIps.remove(clientId);
          ipPool.add(ip);
      }
   }

   public static void main(String[] args) {
      DHCPServer dhcpServer = new DHCPServer();

      String client1Ip = dhcpServer.assignIp("client1");
      System.out.println("Client 1 assigned IP: " + client1Ip);

      String client2Ip = dhcpServer.assignIp("client2");
      System.out.println("Client 2 assigned IP: " + client2Ip);
dhcpServer.releaseIp("client1");
      System.out.println("Client 1 released IP.");
   }
}
```

Output:

Client 1 assigned IP: 192.168.1.10
Client 2 assigned IP: 192.168.1.11
Client 1 released IP.

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result:

DHCP dynamically assigns unique IP addresses to multiple clients and releases them when no longer needed.

Ex. No: 10	Simulation of E-mail system focusing on sending, receiving, and deleting messages.
Date:	

Aim:

To simulate an email system that allows sending, receiving, and deleting messages.

Algorithm:

1. Email Server:
   - Maintain a list of users and their mailboxes.
   - Store sent and received messages.

2. Sending Email:
   - User composes a message with recipient address, subject, and body.
   - Server stores the message in the recipient's mailbox.

3. Receiving Email:
   - User requests to view their mailbox.
   - Server displays the list of received messages.

4. Deleting Email:
   - User selects a message to delete.
   - Server removes the message from the mailbox.

Program:

```
import java.util.*;

class EmailServer {
    private Map<String, List<Map<String, String>>> mailboxes;

    public EmailServer() {
        mailboxes = new HashMap<>();
    }public void sendEmail(String sender, String recipient, String subject, String body) {
        mailboxes.computeIfAbsent(recipient, k -> new ArrayList<>())
                .add(Map.of("sender", sender, "subject", subject, "body", body));
    }
```

```
    public List<Map<String, String>> receiveEmail(String recipient) {
        return mailboxes.getOrDefault(recipient, new ArrayList<>());
    }
    public void deleteEmail(String recipient, int index) {
        if (mailboxes.containsKey(recipient) {
            mailboxes.get(recipient).remove(index);

   }
    }

    public static void main(String[] args) {
        EmailServer server = new EmailServer();
        server.sendEmail("userA", "userB", "Hello", "Hi there!");
        List<Map<String, String>> emails = server.receiveEmail("userB");
        System.out.println("User B's mailbox: " + emails);

        server.deleteEmail("userB", 0);
        System.out.println("User B's mailbox after deletion: " + server.receiveEmail("userB"));
    }
}
```

Output:

```
User B's mailbox: [{sender=userA, subject=Hello, body=Hi there!}]
User B's mailbox after deletion: []
```

Department of IT		
Performance	30	
Observation	30	
Record	40	
Total	100	

Result :

The email system successfully simulates sending, receiving, and deleting messages.